

KURA

Gestão de continuidade veterinária — cliente Clyvo Vet

Documentação Técnica da Sprint 3

Mastering Relational and Non-Relational Database

Entrega: 2 procedimentos, 2 funções, 1 gatilho, estrutura e carga

Disciplina	Mastering Relational and Non-Relational Database
Instituição	FIAP — Challenge 2026
Turma	2TDSPF — 2º ano, Análise e Desenvolvimento de Sistemas
Sistema	KURA — gestão de continuidade veterinária (cliente: Clyvo Vet)
Entrega	Sprint 3 — 2 procedimentos, 2 funções, 1 gatilho, estrutura e carga
Data	2026-09-12

Integrantes (ordem alfabética)

Nome completo	RM
Clayton Alves dos Santos	RM562285
Felipe Ferrete Soares Lemes	RM562999
Guilherme Sola Garcia	RM563674
Gustavo Bosak Santos	RM566315
Nikolas Henrique de Souza Lemes Brisola	RM564371

Sumário

1. Introdução e escopo da entrega
2. Modelo do banco de dados
3. Função 1 — [FN_COBRANCA_JSON](#)
4. Procedimento 1 — [PRC_LISTAR_COBRANCAS_JSON](#)
5. Procedimento 2 — [PRC_RELATORIO_COBRANCAS](#)
6. Função 2 — [FN_CALCULAR_SCORE_URGENCIA](#)
7. Gatilho — [TRG_AUDITORIA_COBRANCA](#) e a tabela [AUDITORIA_COBRANCA](#)
8. Modelo de tratamento de erros
9. Nota sobre o código da 2ª Sprint
10. Uso de Inteligência Artificial
11. Pré-requisitos e como executar
12. Apêndice — mapa dos prints

Este sumário é apresentado **sem numeração de página**: o documento é paginado pelo mecanismo de impressão no momento da exportação, e um número impresso aqui não poderia ser garantido. Sumário sem paginação é preferível a sumário que erra a paginação.

Introdução e escopo da entrega

O KURA é um sistema real de gestão de continuidade veterinária, operado por dois backends independentes (um .NET para a clínica, um Java para o tutor) sobre um **único banco Oracle 19c** compartilhado. O schema é mantido por **19 migrations Flyway** no repositório [backend-tutor-java](#).

O arquivo de entrega `2TDS PF_2026_CodigoSql_Integrantes.sql` é a **consolidação dessas 19 migrations num script único e executável** — cada tabela nasce no estado final, com todas as ~75 alterações posteriores (`ALTER TABLE`) já aplicadas — **mais** os 5 objetos PL/SQL e a tabela de auditoria exigidos pela Sprint 3.

1.1. O que a Sprint 3 pede e onde está no arquivo

Item	Pts	Onde
Procedimento 1 — JOIN ≥ 2 tabelas, saída JSON string por função do grupo, ≥ 3 exceções	15	Bloco 9.1
Procedimento 2 — tabela de fatos, soma por combinação + subtotal + total geral, sem ROLLUP/CUBE/GROUPING , formato fixo, ≥ 3 exceções	15	Bloco 9.2
Função 1 — relacional → JSON string, lógica de conversão do grupo, sem built-in de JSON, ≥ 3 exceções	15	Bloco 8.1
Função 2 — substitui um processo lógico do projeto, ≥ 3 exceções	15	Bloco 8.2
Gatilho — tabela de auditoria (usuário, operação, data/hora, :OLD , :NEW) + trigger AFTER INSERT OR UPDATE OR DELETE	30	Blocos 6 e 10
Entrega e documentação (este PDF + o <code>.sql</code>)	10	—

1.2. A espinha: tudo gira em torno de COBRANCA

Os cinco objetos **não são cinco exercícios isolados** — são um recorte do sistema:

- `FN_COBRANCA_JSON` serializa uma cobrança em JSON, à mão.
- `PRC_LISTAR_COBRANCAS_JSON` consome essa função para expor cobranças (JOIN de 3 tabelas).
- `TRG_AUDITORIA_COBRANCA` reusa a **mesma função** para gravar **:OLD**/**:NEW** na trilha de auditoria.
- `PRC_RELATORIO_COBRANCAS` agrega a **mesma tabela** que a trigger audita.
- `FN_CALCULAR_SCORE_URGENCIA` traz para o banco a regra de urgência que hoje só existe no serviço de IA (Luna).

COBRANCA foi escolhida como tabela de fatos porque `VL_COBRADO` é `NUMBER(10,2)` (dinheiro com 2 casas) — uma soma "manual" precisa parecer uma soma manual, e não um `COUNT(*) × k` — e porque **COBRANCA não tem nenhuma coluna de texto livre**, então auditá-la não cria superfície de vazamento de dado pessoal (ao contrário de auditar **AGENDAMENTO**, que guarda observação escrita pelo tutor).

1.3. Escopo declarado

- O `.sql` cria **todas as 30 tabelas** do schema no estado da migration V19, **29 sequences**, **13 índices**, **2 views** e a tabela `AUDITORIA_COBRANCA` (objeto acadêmico da Sprint 3, que **não** vira migration Flyway).
- A carga tem **≥ 5 registros por tabela** (ruling do time). Dados **fictícios**, declarados como tais no comentário do bloco 7 (e-mails em domínios reservados `.example`, CPF/CNPJ/telefones inventados, hashes com forma de BCrypt mas sem valor).

- O schema Oracle da FIAP é **compartilhado entre disciplinas**; por isso o bloco de limpeza do `.sql` derruba **apenas** os objetos do KURA, por nome. A conferência de "todas as tabelas" é feita por nome (`WHERE table_name IN (...)`), não por `SELECT COUNT(*) FROM user_tables`.

Modelo do banco de dados

2.1. As 30 tabelas (estado da migration V19)

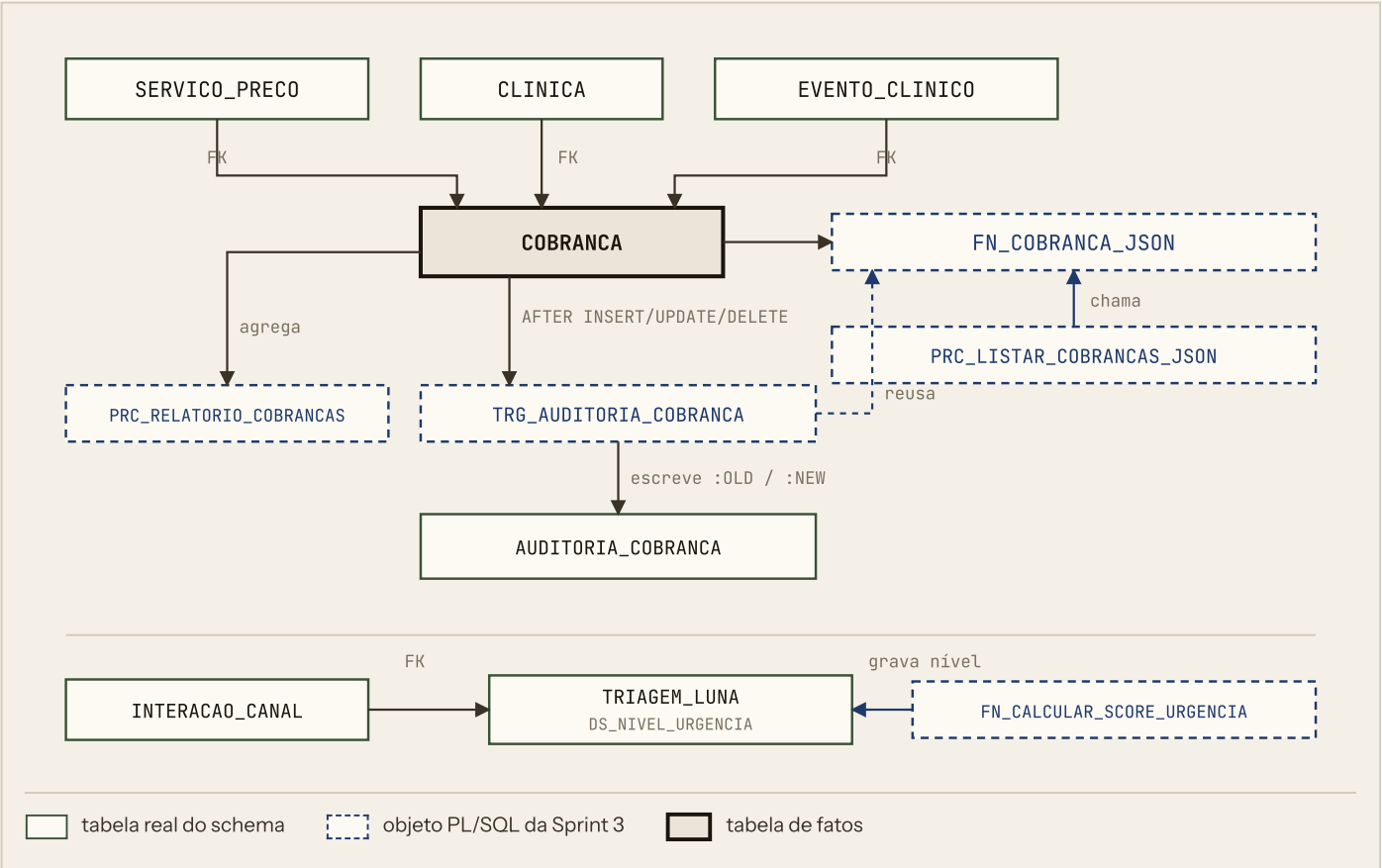
Ordenadas por nível de dependência de chave estrangeira:

Nível	Tabelas
1	CLINICA, ESPECIE, TIPO_EVENTO, MEDICAMENTO, LOG_ERRO, IDEMPOTENCY_KEY
2	RACA, VETERINARIO, TUTOR, DISPOSITIVO_IOT, SERVICIO_PRECO
3	PET, INVITE_TUTOR, CONSENTIMENTO, USUARIO_CLINICA, LEITURA_TEMPERATURA, INTERACAO_CANAL, NOTIFICACAO
4	TUTOR_PET, CONTA_TUTOR, EVENTO_CLINICO, ALERTA_TEMPERATURA, TRIAGEM_LUNA
5	AGENDAMENTO, CONSULTA, EXAME, VACINA, PRESCRICAO, DOCUMENTO, COBRANCA

Estratégia de chave primária (herdada do banco real, não uniformizada):

- **26 tabelas** usam `DEFAULT SEQ_X.NEXTVAL` (padrão do backend .NET).
- **4 tabelas** usam `GENERATED BY DEFAULT AS IDENTITY` (padrão do backend Java): `CONTA_TUTOR`, `CONSENTIMENTO`, `IDEMPOTENCY_KEY` e `AGENDAMENTO`.
- `AGENDAMENTO` é `IDENTITY` no banco e puxa de `SEQ_AGENDAMENTO` pela entidade JPA — as duas coisas coexistem porque `GENERATED BY DEFAULT` (diferente de `GENERATED ALWAYS`) aceita valor explícito. Não é lixo; é a realidade do banco, e está documentada no `.sql`.

2.2. O recorte usado pelos 5 objetos



Recorte do schema usado pelos 5 objetos da Sprint 3. Setas cheias = chave estrangeira ou escrita; setas tracejadas = chamada de função. Apenas as tabelas efetivamente tocadas pelos objetos estão representadas — as outras 24 do schema V19 estão listadas em 2.1.

2.3. Convenções Oracle do projeto

- Booleanos:** `CHAR(1)` com 'S' / 'N' + `CHECK`.
- Soft delete:** coluna `ST_ATIV*` — `DELETE` físico nunca acontece no produto (a trigger da Sprint 3 audita `DELETE` mesmo assim, para a demonstração).
- Datas:** `TIMESTAMP DEFAULT CURRENT_TIMESTAMP`.

FN_COBRANCA_JSON

3.1. O que faz

Recebe os **campos escalares de uma cobrança** e devolve uma **string JSON válida**, construída 100% à mão (concatenação + escape próprio). **Não usa** `JSON_OBJECT`, `JSON_ARRAY`, `TO_JSON`, `JSON_TABLE` nem similar.

3.2. Por que existe no KURA

É a peça que o Procedimento 1 consome para expor cobranças e que a Trigger reusa para gravar `:OLD/:NEW`. Ter **um** serializador do grupo, e não três, é o que amarra a entrega numa história só.

3.3. Por que a assinatura é escalar

A trigger `FOR EACH ROW` sobre `COBRANCA` **não pode consultar** `COBRANCA` (`ORA-04091`, tabela mutante). Por isso a função recebe os 8 valores escalares e só consulta `SERVICO_PRECO` (que a trigger não toca). Se a assinatura recebesse só `p_id_cobranca` e buscasse o resto por `SELECT`, a trigger quebraria em runtime — e compilaria normalmente.

3.4. Os 4 pontos técnicos

1. **Escape manual, e a ordem importa:** troca-se `\` por `\\` antes de `"` por `\`". Na ordem inversa, a barra invertida que o escape da aspa insere seria re-escapada e o JSON sairia corrompido.
2. **NULL vira o literal `null`, sem aspas** — não string vazia. `ID_SERVICO_PRECO` e `DS_FORMA_PAGAMENTO` são nullable, então esse caminho é exercitado de verdade.
3. **Número com ponto decimal forçado:** a sessão da FIAP é pt-BR (`NLS_NUMERIC_CHARACTERS = ',.'`), então `TO_CHAR(12.5)` daria `"12,5"` e `{"valor": 12,5}` é **JSON inválido**. Força-se `NLS_NUMERIC_CHARACTERS='.'` no `TO_CHAR`.
4. **Data em formato estável:** `TO_CHAR(p_dt_cobranca, 'YYYY-MM-DD"T"HH24:MI:SS')`, nunca o default de `NLS_TIMESTAMP_FORMAT`.

3.5. Exceções (3)

Handler	Quando dispara	O que faz
<code>NO_DATA_FOUND</code>	<code>p_id_servico</code> informado mas inexistente em <code>SERVICO_PRECO</code>	devolve <code>{"erro": "NO_DATA_FOUND", ...}</code>
<code>VALUE_ERROR</code>	estouro do <code>VARCHAR2</code> de retorno / conversão inválida	devolve <code>{"erro": "VALUE_ERROR", ...}</code>
<code>OTHERS</code>	qualquer outra falha	devolve <code>{"erro": "OTHERS", "sqlcode": ...}</code>

MOTIVO ESTRUTURAL

O `WHEN OTHERS` é **obrigatório por motivo estrutural**: a trigger chama esta função. Se ela propagar exceção, derruba o `INSERT/UPDATE/DELETE` que disparou a trigger. Ela **sempre** devolve um JSON de fallback, **nunca** propaga. E **não grava** em `LOG_ERRO` — `COMMIT` dentro de trigger é `ORA-04092`.

3.6. Prints

```
##### BLOCO 11: DEMONSTRACAO #####

==== 11.1 FN_COBRANCA_JSON -- caminho feliz (aspas escapadas e servico nulo) ====
cobranca 8 (servico com aspas): {"idCobranca":8,"idEvento":4,"idClinica":1,"idServico":3,"servico":"Banho e Tosa \"Premium\"",\"valor\":132.75,\"formaPagamento\":\"PIX\", \"dataCobranca\":\"2026-08-28T21:14:53\", \"ativa\":true}
cobranca 3 (servico nulo)      : {"idCobranca":3,\"idEvento\":3,\"idClinica\":1,\"idServico\":null,\"servico\":null,\"valor\":235.90,\"formaPagamento\":\"CARTAO_CREDITO\", \"dataCobranca\":\"2026-08-23T21:14:53\", \"ativa\":true}
```

Print 1 — bloco 11.1· caminho feliz. Prova que a função serializa uma cobrança em JSON válido à mão: escape de aspas, `null` literal e ponto decimal em sessão pt-BR. Arquivo: 01_fn_cobranca_json_sucesso.png

Saída (bloco 11.1):

```
cobranca 8 (servico com aspas): {"idCobranca":8,"idEvento":4,"idClinica":1,"idServico":3,
  "servico":"Banho e Tosa \"Premium\"",\"valor\":132.75,\"formaPagamento\":\"PIX\",
  \"dataCobranca\":\"2026-08-28T...\", \"ativa\":true}
cobranca 3 (servico nulo)      : {"idCobranca":3,...,\"idServico\":null,\"servico\":null,
  \"valor\":235.90,\"formaPagamento\":\"CARTAO_CREDITO\", ...}
```

- `\"Premium\"` — aspas escapadas corretamente.
- `"idServico":null,"servico":null` — `NULL` como literal, sem aspas.
- `"valor":132.75` — ponto decimal, apesar da sessão pt-BR.

```
==== 11.2 FN_COBRANCA_JSON -- excecoes tratadas ====
[FN_COBRANCA_JSON] NO_DATA_FOUND: servico 999 inexistente.
NO_DATA_FOUND -> {"erro":"NO_DATA_FOUND","idCobranca":1,"detalhe":"servico inexistente"}
[FN_COBRANCA_JSON] VALUE_ERROR: ORA-06502: PL/SQL: erro: buffer de string de caracteres pequeno demais numérico ou de valor
VALUE_ERROR   -> {"erro":"VALUE_ERROR","idCobranca":2}
```

Print 2 — bloco 11.2· exceção tratada. Prova que `NO_DATA_FOUND` e `VALUE_ERROR` são capturados e devolvem JSON de fallback, sem propagar. Arquivo: 02_fn_cobranca_json_excecoes.png

Saída (bloco 11.2):

```
NO_DATA_FOUND -> {"erro":"NO_DATA_FOUND","idCobranca":1,"detalhe":"servico inexistente"}
VALUE_ERROR   -> {"erro":"VALUE_ERROR","idCobranca":2}
```

A função tratou as duas exceções e devolveu JSON de fallback — **não propagou**.

PRC_LISTAR_COBRANCAS_JSON

4.1. O que faz

Percorre um **cursor explícito com JOIN de 3 tabelas** (COBRANCA + CLINICA + LEFT JOIN SERVICO_PRECO), chama **FN_COBRANCA_JSON** por linha e imprime **um objeto JSON por linha** com **DBMS_OUTPUT.PUT_LINE**, agrupado por clínica.

- O **LEFT JOIN** em **SERVICO_PRECO** é proposital — **ID_SERVICO_PRECO** é nullable.
- **Um JSON por linha** porque **DBMS_OUTPUT.PUT_LINE** aborta acima de 32.767 bytes por linha.
- Parâmetro **p_id_clinica** (default **NULL** = todas as clínicas).

4.2. Por que 3 tabelas e não 2

A rubrica pede 2 ou mais. **COBRANCA** → **CLINICA** → **SERVICO_PRECO** é o JOIN que o relatório financeiro real do sistema usa: cobrança + nome da clínica + nome do serviço.

4.3. Exceções (3)

NO_DATA_FOUND (clínica do filtro inexistente/inativa, ou nenhuma cobrança), **VALUE_ERROR**, **OTHERS**. Padrão de tratamento herdado da 2ª Sprint (**banco/kura_req1**): **ROLLBACK** → **INSERT INTO LOG_ERRO** → **COMMIT** → **DBMS_OUTPUT**. Como é chamada **direta** (transação própria), pode gravar em **LOG_ERRO** — ao contrário de **FN_COBRANCA_JSON**, que a trigger chama.

4.4. Prints

```
==== 11.3 PRC_LISTAR_COBRANCAS_JSON -- caminho feliz (todas as clinicas) ====
PRC_LISTAR_COBRANCAS_JSON (clinica: TODAS) =====
-- Clinica: Clinica AnimalCare Zona Sul (id 2) --
[Consulta Especializada] {"idCobranca":11,"idEvento":5,"idClinica":2,"idServico":4,"servico":"Consulta Especializada","valor":158.90,"formaPagamento":"DINHEIRO","dataCobranca":"2026-08-31T21:14:53","ativa":true}
[(avulso)] {"idCobranca":12,"idEvento":7,"idClinica":2,"idServico":null,"servico":null,"valor":121.30,"formaPagamento":"DINHEIRO","dataCobranca":"2026-09-01T21:14:54","ativa":true}
[Consulta Especializada] {"idCobranca":13,"idEvento":5,"idClinica":2,"idServico":4,"servico":"Consulta Especializada","valor":242.75,"formaPagamento":"CARTAO_CREDITO","dataCobranca":"2026-09-02T21:14:54","ativa":true}
[Exame de Sangue Completo] {"idCobranca":14,"idEvento":6,"idClinica":2,"idServico":5,"servico":"Exame de Sangue Completo","valor":195.40,"formaPagamento":"CARTAO_CREDITO","dataCobranca":"2026-09-03T21:14:54","ativa":true}
[Exame de Sangue Completo] {"idCobranca":15,"idEvento":6,"idClinica":2,"idServico":5,"servico":"Exame de Sangue Completo","valor":133.60,"formaPagamento":"PIX","dataCobranca":"2026-09-04T21:14:54","ativa":true}
[Limpeza Dentaria] {"idCobranca":16,"idEvento":7,"idClinica":2,"idServico":6,"servico":"Limpeza Dentaria","valor":287.20,"formaPagamento":"PIX","dataCobranca":"2026-09-05T21:14:54","ativa":true}
[(avulso)] {"idCobranca":17,"idEvento":5,"idClinica":2,"idServico":null,"servico":null,"valor":104.85,"formaPagamento":"TRANSFERENCIA","dataCobranca":"2026-09-06T21:14:54","ativa":true}
[Limpeza Dentaria] {"idCobranca":18,"idEvento":6,"idClinica":2,"idServico":6,"servico":"Limpeza Dentaria","valor":218.05,"formaPagamento":"TRANSFERENCIA","dataCobranca":"2026-09-07T21:14:54","ativa":true}
-- Clinica: Clinica PetVida Centro (id 1) --
[Consulta Geral] {"idCobranca":1,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":147.30,"formaPagamento":"DINHEIRO","dataCobranca":"2026-08-21T21:14:53","ativa":true}
[Vacina V10] {"idCobranca":2,"idEvento":2,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":92.85,"formaPagamento":"DINHEIRO","dataCobranca":"2026-08-22T21:14:53","ativa":true}
[(avulso)] {"idCobranca":3,"idEvento":3,"idClinica":1,"idServico":null,"servico":null,"valor":235.90,"formaPagamento":"CARTAO_CREDITO","dataCobranca":"2026-08-23T21:14:53","ativa":true}
[Consulta Geral] {"idCobranca":4,"idEvento":4,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":168.40,"formaPagamento":"CARTAO_CREDITO","dataCobranca":"2026-08-24T21:14:53","ativa":true}
[Vacina V10] {"idCobranca":5,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":119.75,"formaPagamento":"CARTAO_DEBITO","dataCobranca":"2026-08-25T21:14:53","ativa":true}
[Consulta Geral] {"idCobranca":6,"idEvento":2,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":204.60,"formaPagamento":"CARTAO_DEBITO","dataCobranca":"2026-08-26T21:14:53","ativa":true}
[(avulso)] {"idCobranca":7,"idEvento":3,"idClinica":1,"idServico":null,"servico":null,"valor":76.80,"formaPagamento":"PIX","dataCobranca":"2026-08-27T21:14:53","ativa":true}
[Banho e Tosa "Premium"] {"idCobranca":8,"idEvento":4,"idClinica":1,"idServico":3,"servico":"Banho e Tosa \"Premium\"","valor":132.75,"formaPagamento":"PIX","dataCobranca":"2026-08-28T21:14:53","ativa":true}
[Consulta Geral] {"idCobranca":9,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":289.15,"formaPagamento":"BOLETO","dataCobranca":"2026-08-29T21:14:53","ativa":true}
[Vacina V10] {"idCobranca":10,"idEvento":2,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":97.60,"formaPagamento":"BOLETO","dataCobranca":"2026-08-30T21:14:53","ativa":true}
===== 18 cobranca(s) serializada(s) =====
```

Print 3 — bloco 11.3·execução. Prova o JOIN de 3 tabelas exibido através da Função 1: 18 objetos JSON, agrupados por -- Clinica: <nome> --, cada linha prefixada pelo nome do serviço ([Consulta Geral], [(avulso)]) quando ID_SERVICO_PRECO é nulo. Arquivo: 03_prc_listar_cobrancas_json.png

```
===== 11.4 PRC_LISTAR_COBRANCAS_JSON -- execucao tratada (clinica 999 -> LOG_ERRO) =====
[PRC_LISTAR_COBRANCAS_JSON] NO_DATA_FOUND -- registrado em LOG_ERRO.
```

Print 4 — bloco 11.4·exceção tratada. Prova o handler NO_DATA_FOUND com registro persistido em LOG_ERRO. Arquivo: 04_prc_listar_json_excecao.png

Saída (bloco 11.4):

```
[PRC_LISTAR_COBRANCAS_JSON] NO_DATA_FOUND -- registrado em LOG_ERRO.
```

PRC_RELATORIO_COBRANCAS

5.1. O que faz

Lê a tabela de fatos **COBRANCA** e imprime um relatório totalizado com **três níveis**:

1. **soma por combinação completa** das categóricas (**ID_CLINICA**, **DS_FORMA_PAGAMENTO**);
2. **Sub Total** por grupo da 1ª categoria (**ID_CLINICA**);
3. **Total Geral** ao final.

5.2. Soma 100% manual

O cursor traz **linhas de detalhe** (cada **COBRANCA**), **sem GROUP BY**, **sem SUM()**, **sem nenhuma função de agregação**, e **sem ROLLUP/CUBE/GROUPING SETS/GROUPING** (proibidos pela rubrica). Toda a totalização é feita em **3 acumuladores PL/SQL** (**v_soma_comb**, **v_sub_clinica**, **v_total_geral**), por **quebra de grupo**.

5.3. As duas quebras de grupo e o último grupo

- Ao mudar **DS_FORMA_PAGAMENTO** (ou **ID_CLINICA**): emite a linha da combinação acumulada.
- Ao mudar **ID_CLINICA**: emite a linha **Sub Total** da clínica anterior.

ARMADILHA

O **último grupo é emitido fora do loop** — não existe próxima iteração para detectar a quebra da última combinação / do último subtotal. Depois do **LOOP**, emitem-se a última combinação, o último **Sub Total** e o **Total Geral**.

5.4. Formato de saída fixo

- Larguras fixas, **ASCII puro** (nada de caixas, blocos ou caracteres de desenho, que viram mojibake em print).
- A **coluna numérica termina no mesmo offset de caractere** nas três formas de linha (detalhe, **Sub Total**, **Total Geral**).
- Nas linhas **Sub Total** / **Total Geral**, as colunas categóricas ficam **literalmente vazias** — o texto **Sub Total**/**Total Geral** é um rótulo, não há valor de agrupamento.

5.5. Exceções (3)

NO_DATA_FOUND, **VALUE_ERROR**, **OTHERS**. Mesmo padrão **kura_req1** do Procedimento 1.

5.6. Prints

```
==== 11.5  PRC_RELATORIO_COBRANCAS -- caminho feliz (formato fixo p.26) ====
==== PRC_RELATORIO_COBRANCAS (clinica: TODAS) ====
Clinica  Forma                Valor
-----
      1  BOLETO                386.75
      1  CARTAO_CREDITO        404.30
      1  CARTAO_DEBITO         324.35
      1  DINHEIRO              240.15
      1  PIX                   209.55
Sub Total                1565.10
      2  CARTAO_CREDITO        438.15
      2  DINHEIRO              280.20
      2  PIX                   420.80
      2  TRANSFERENCIA         322.90
Sub Total                1462.05
Total Geral              3027.15
==== 18 lancamento(s) totalizado(s) ====
```

Print 5 — bloco 11.5·execução no formato exigido. Prova subtotais e total geral na mesma coluna da saída, com as colunas de agrupamento ausentes nas linhas de Sub Total e Total Geral. Arquivo: 05_prc_relatorio_cobrancas.png

Saída (bloco 11.5) — transcrição literal, ASCII monoespado, alinhamento preservado:

Clinica	Forma	Valor
-----	-----	-----
1	BOLETO	386.75
1	CARTAO_CREDITO	404.30
1	CARTAO_DEBITO	324.35
1	DINHEIRO	240.15
1	PIX	209.55
Sub Total		1565.10
2	CARTAO_CREDITO	438.15
2	DINHEIRO	280.20
2	PIX	420.80
2	TRANSFERENCIA	322.90
Sub Total		1462.05
Total Geral		3027.15

Conferência aritmética por fora (um **SELECT** com **SUM/GROUP BY**, não incluído no **.sql** de entrega): os subtotais e o total geral batem exatamente.

```
==== 11.6  PRC_RELATORIO_COBRANCAS -- excecao tratada (clinica 999 -> LOG_ERRO) ====
[PRC_RELATORIO_COBRANCAS] NO_DATA_FOUND -- registrado em LOG_ERRO.
```

Print 6 — bloco 11.6·exceção tratada. Prova o handler **NO_DATA_FOUND** do Procedimento 2, com registro em **LOG_ERRO**. Arquivo: 06_prc_relatorio_excecao.png

Saída (bloco 11.6):

```
[PRC_RELATORIO_COBRANCAS] NO_DATA_FOUND -- registrado em LOG_ERRO.
```

FN_CALCULAR_SCORE_URGENCIA

6.1. Que processo do projeto ela substitui

O **motor de triagem léxico da Luna**, que roda hoje em produção: `kura-luna-ai/luna/src/ai/triage_engine.py` (`TriageEngine.classificar`) + `luna/src/ai/triage_rules.py` (`TRIAGE_RULES_VERSION = "1.0"`). É o único candidato em que a frase *"substitui um processo lógico do projeto"* é verdadeira e ancorável em **arquivo:linha**. Trazer a regra para o banco permite reclassificar histórico e classificar na própria carga, sem subir o serviço Python.

6.2. O algoritmo (portado 1:1)

- Pontuação: **ALTA = 10**, **MEDIA = 3**, **BAIXA = 1**.
- 3 níveis, nesta ordem: **ALTA** (5 categorias de sintoma), **MEDIA** (4), **BAIXA** (2).
- O texto é **normalizado** (minúsculas + remoção de acento) — e **as palavras-chave também**, com a mesma função. As keywords são escritas já normalizadas no `.sql`.
- Casamento por **substring** (`INSTR > 0`), **sem fronteira de palavra** — igual ao `if _normalize(kw) in normalized_text` do Python.
- Cada **categoria** conta **uma vez** por nível (o `break` do Python).
- O **score acumula os pontos de todos os níveis com match** — não só o vencedor: um texto com 1 categoria ALTA + 1 MEDIA + 1 BAIXA pontua $10 + 3 + 1 = 14$.
- O **nível retornado** é o **primeiro nível** (na ordem ALTA > MEDIA > BAIXA) com qualquer match. Sem match nenhum, ou texto vazio/nulo $\Rightarrow$ **'BAIXA'**.
- O nível calculado é gravado em `TRIAGEM_LUNA.DS_NIVEL_URGENCIA`.

6.3. Limites herdados (perguntas de banca)

- **Sem fronteira de palavra:** *"veneno"* é substring de *"venenoso"* $\rightarrow$ um texto sobre "remédio venenoso" classifica como ALTA. Falso positivo herdado do motor original.
- **Sem tratamento de negação:** *"ele não está vomitando"* classifica como MEDIA — o **não** é ignorado.

Esses limites são **conhecidos e documentados**, não descobertos pelo avaliador. A escolha de um motor léxico determinístico (em vez de um modelo) é defensável: auditável, explicável, custo zero, sem risco de alucinação em contexto clínico.

6.4. Exceções (3)

Handler	Quando dispara
<code>e_texto_excede_limite</code> (exceção do grupo)	texto acima de 4000 caracteres — a <i>verificação de limites</i> que a rubrica cita para a Função 2
<code>VALUE_ERROR</code>	estouro de buffer / conversão inválida em operação de string
<code>OTHERS</code>	qualquer outra falha

Em erro: grava `LOG_ERRO` (chamada direta, não é chamada por trigger) e **retorna 'BAIXA'** — degrada para o nível mais baixo, que um humano revisa, já que a coluna destino é `NOT NULL`.

COMO CHAMAR

Porque ela grava DML no tratamento de erro, esta função deve ser chamada **de dentro de PL/SQL** (atribuição a variável), **nunca** embutida direto num **SELECT** nem no **SET** de um **UPDATE** — ali o Oracle proíbe DML (**ORA-14551**). O bloco de demonstração classifica num loop PL/SQL e só então faz o **UPDATE** em **TRIAGEM_LUNA**.

6.5. Prints

```
==== 11.7  FN_CALCULAR_SCORE_URGENCIA -- >= 5 casos (um de cada nivel + vazio) ====
[FN_CALCULAR_SCORE_URGENCIA] score=20 -> nivel=ALTA
  nivel=ALTA  (esperado ALTA ) | Meu cachorro teve uma convulsao e esta sangrando muito
[FN_CALCULAR_SCORE_URGENCIA] score=6 -> nivel=MEDIA
  nivel=MEDIA (esperado MEDIA ) | A gata esta vomitando e com diarreia desde ontem
[FN_CALCULAR_SCORE_URGENCIA] score=1 -> nivel=BAIXA
  nivel=BAIXA (esperado BAIXA ) | Tenho uma duvida sobre a racao do filhote
[FN_CALCULAR_SCORE_URGENCIA] score=0 -> nivel=BAIXA
  nivel=BAIXA (esperado BAIXA ) | Ele esta otimo, so passei para agradecer o atendimento
  nivel=BAIXA (esperado BAIXA ) | (NULO)
[FN_CALCULAR_SCORE_URGENCIA] score=14 -> nivel=ALTA
  nivel=ALTA  (esperado ALTA ) | meu cao teve convulsao, esta com febre e uma duvida
```

Print 7 — bloco 11.7 • execução. Prova a classificação nos três níveis, o caso de texto vazio e a acumulação de score entre níveis. Arquivo: 07_fn_calcular_score_urgencia.png

Saída (bloco 11.7):

```
score=20 -> nivel=ALTA  | Meu cachorro teve uma convulsao e esta sangrando muito
score=6   -> nivel=MEDIA | A gata esta vomitando e com diarreia desde ontem
score=1   -> nivel=BAIXA | Tenho uma duvida sobre a racao do filhote
score=0   -> nivel=BAIXA | Ele esta otimo, so passei para agradecer o atendimento
           nivel=BAIXA  | (texto vazio)
score=14  -> nivel=ALTA  | meu cao teve convulsao, esta com febre e uma duvida
```

O último caso demonstra a acumulação: **10 (ALTA) + 3 (MEDIA) + 1 (BAIXA) = 14**, mas o nível retornado é o mais grave (**ALTA**).

Equivalência com o motor original: a função foi validada contra uma reimplementação independente do **classificar()** usando as palavras-chave literais do **triage_rules.py** — **7/7 casos idênticos**.

```
(o ultimo caso tem score=14 = 10+3+1: ALTA+MEDIA+BAIXA acumulam, mas o nivel e o mais grave)

==== 11.8  FN_CALCULAR_SCORE_URGENCIA -- excecao tratada (texto acima do limite -> LOG_ERRO) ====
[FN_CALCULAR_SCORE_URGENCIA] TEXTO_EXCEDE_LIMITE -- registrado em LOG_ERRO. Retorna BAIXA.
  retornou (apos tratar): BAIXA
```

Print 8 — bloco 11.8 • exceção tratada. Prova a exceção de grupo e_texto_excede_limite: registro em LOG_ERRO e degradação para BAIXA. Arquivo: 08_fn_score_urgencia_excecao.png

```
[FN_CALCULAR_SCORE_URGENCIA] TEXTO_EXCEDE_LIMITE -- registrado em LOG_ERRO. Retorna BAIXA.
```

```

===== 11.9 FN_CALCULAR_SCORE_URGENCIA -- UPDATE real em TRIAGEM_LUNA.DS_NIVEL_URGENCIA =====
(a funcao classifica DS_DESCRICAO e o nivel calculado e gravado na coluna. Chamada de
dentro de PL/SQL -- nao embutida no SET do UPDATE -- porque a funcao grava LOG_ERRO.)
[FN_CALCULAR_SCORE_URGENCIA] score=20 -> nivel=ALTA
    triagem 1: ALTA    -> ALTA    | Pet com convulsao e sangramento intenso, chegando agora
[FN_CALCULAR_SCORE_URGENCIA] score=6 -> nivel=MEDIA
    triagem 2: MEDIA   -> MEDIA   | Animal vomitando muito e com febre desde ontem
[FN_CALCULAR_SCORE_URGENCIA] score=1 -> nivel=BAIXA
    triagem 3: BAIXA   -> BAIXA   | Tenho uma duvida sobre a alimentacao, ele esta bem
[FN_CALCULAR_SCORE_URGENCIA] score=0 -> nivel=BAIXA
    triagem 4: MEDIA   -> BAIXA   | Acompanhamento de funcao renal. Retorno em 30 dias.
[FN_CALCULAR_SCORE_URGENCIA] score=1 -> nivel=BAIXA
    triagem 5: BAIXA   -> BAIXA   | Duvida geral sobre alimentacao respondida pela Luna.

```

Print 13 — bloco 11.9 • aplicação no schema. Prova que o nível calculado é gravado em TRIAGEM_LUNA.DS_NIVEL_URGENCIA — a função substitui, de fato, o processo lógico do projeto. Arquivo: 13_triagem_luna_update.png

Saída (bloco 11.9):

```

triagem 1: ALTA    → ALTA    | Pet com convulsao e sangramento intenso, chegando agora
triagem 2: MEDIA   → MEDIA   | Animal vomitando muito e com febre desde ontem
triagem 3: BAIXA   → BAIXA   | Tenho uma duvida sobre a alimentacao, ele esta bem
triagem 4: MEDIA   → BAIXA   | Acompanhamento de funcao renal. Retorno em 30 dias.
triagem 5: BAIXA   → BAIXA   | Duvida geral sobre alimentacao respondida pela Luna.

```

A triagem 4 foi **reclassificada** (MEDIA → BAIXA) — a descrição não contém nenhuma palavra-chave do motor.

==== 11.11 AUDITORIA_COBRANCA -- trilha com :OLD e :NEW lado a lado ====				
ID_AUDITORIA	NM_USUARIO	DS_OPERACA	QUANDO	ID_COBRANCA VALORES_OLD
1	RM562285	INSERT	2026-09-08 21:15:13	950 (nulo)
2	RM562285	INSERT	2026-09-08 21:15:13	951 (nulo)
3	RM562285	INSERT	2026-09-08 21:15:13	952 (nulo)
4	RM562285	UPDATE	2026-09-08 21:15:13	950 {"idCobranca":950,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":111.11,"formaPagamento":"P
5	RM562285	UPDATE	2026-09-08 21:15:13	951 {"idCobranca":951,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":222.22,"formaPagamento":"CARTA
6	RM562285	DELETE	2026-09-08 21:15:13	950 {"idCobranca":950,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":150.00,"formaPagamento":"B
7	RM562285	DELETE	2026-09-08 21:15:13	951 {"idCobranca":951,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":222.22,"formaPagamento":"CARTA
8	RM562285	DELETE	2026-09-08 21:15:13	952 {"idCobranca":952,"idEvento":1,"idClinica":1,"idServico":null,"servico":null,"valor":333.33,"formaPagamento":"DINHEIRO",
8 linhas selecionadas.				

Print 11 (a) — blocos 11.10/11.11 (DELETE), coluna DS_VALORES_OLD. Prova que o estado anterior da linha excluída foi preservado.

Arquivo: 11_trigger_delete_old.png

ID_COBRANCA VALORES_OLD	VALORES_NEW
9-08 21:15:13 950 (nulo)	{"idCobranca":950,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":111.11,"formaPagamento":"P
9-08 21:15:13 951 (nulo)	{"idCobranca":951,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":222.22,"formaPagamento":"CARTA
9-08 21:15:13 952 (nulo)	{"idCobranca":952,"idEvento":1,"idClinica":1,"idServico":null,"servico":null,"valor":333.33,"formaPagamento":"DINHEIRO",
9-08 21:15:13 950 {"idCobranca":950,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":111.11,"formaPagamento":"P	{"idCobranca":950,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":150.00,"formaPagamento":"B
9-08 21:15:13 951 {"idCobranca":951,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":222.22,"formaPagamento":"CARTA	{"idCobranca":951,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":222.22,"formaPagamento":"CARTA
9-08 21:15:13 950 {"idCobranca":950,"idEvento":1,"idClinica":1,"idServico":1,"servico":"Consulta Geral","valor":150.00,"formaPagamento":"B	(nulo)
9-08 21:15:13 951 {"idCobranca":951,"idEvento":1,"idClinica":1,"idServico":2,"servico":"Vacina V10","valor":222.22,"formaPagamento":"CARTA	(nulo)
9-08 21:15:13 952 {"idCobranca":952,"idEvento":1,"idClinica":1,"idServico":null,"servico":null,"valor":333.33,"formaPagamento":"DINHEIRO",	(nulo)

Print 11 (b) — blocos 11.10/11.11 (DELETE), coluna DS_VALORES_NEW. Mesma linha de auditoria do print 11 (a), mostrando DS_VALORES_NEW

nulo. Arquivo: 11_trigger_delete_new.png

Saída (blocos 11.10 e 11.11) — trilha AUDITORIA_COBRANCA após 3 INSERT + 2 UPDATE + 3 DELETE em COBRANCA:

ID	USUARIO	OPERACAO	ID_COBRANCA	DS_VALORES_OLD	DS_VALORES_NEW
1	RM_-----	INSERT	950	(nulo)	{ ... , "valor":111.11, ... }
2	RM_-----	INSERT	951	(nulo)	{ ... , "valor":222.22, ... }
3	RM_-----	INSERT	952	(nulo)	{ ... , "servico":null, ... }
4	RM_-----	UPDATE	950	{ ... 111.11 ... }	{ ... 150.00, "BOLETO" ... }
5	RM_-----	UPDATE	951	{ ... 222.22 ... }	{ ... "ativa":false }
6	RM_-----	DELETE	950	{ ... 150.00 ... }	(nulo)
7	RM_-----	DELETE	951	{ ... 222.22 ... }	(nulo)
8	RM_-----	DELETE	952	{ ... 333.33 ... }	(nulo)


```

===== 11.12  CONTAGENS FINAIS =====
(AUDITORIA_COBRANCA >= 5 ; as 30 tabelas do V19 com COUNT(*) >= 5)

DS_OPERACA      QTD
-----
DELETE          3
INSERT          3
UPDATE          2

3 linhas selecionadas.

TOTAL_AUDITORIA
-----
                8

1 linha selecionada.

MENOR_CONTAGEM_DAS_30_TABELAS_V19
-----
                                5

1 linha selecionada.

```

Print 12 — bloco 11.12·consolidação. Prova as 8 operações DML auditadas por tipo e a menor contagem das 30 tabelas.
Arquivo: 12_auditoria_e_contagem.png

Saída (bloco 11.12):

DS_OPERACAO	QTD	TOTAL_AUDITORIA	MENOR_CONTAGEM_DAS_30_TABELAS_V19
DELETE	3	8	5
INSERT	3		
UPDATE	2		

7.4. Prova de que foi a trigger que gravou (mutação)

Com o teste independente [SD-07-teste.sql](#): a trigger foi **desabilitada** (`ALTER TRIGGER TRG_AUDITORIA_COBRANCA DISABLE`), as 3 operações foram repetidas, e **nenhuma linha** foi gravada em `AUDITORIA_COBRANCA`. Reabilitada, voltou a gravar.

Modelo de tratamento de erros

Padrão herdado da 2ª Sprint (banco/kura_req1_procedures_carga.sql): cada handler faz **ROLLBACK** → **INSERT INTO LOG_ERRO (... , SQLCODE, SQLERRM, DS_PARAMETROS)** → **COMMIT** → **DBMS_OUTPUT**. Assim o "print de exceção" deixa de ser texto no console e passa a ser **linha persistida** que dá para consultar.

Objeto	Exceção 1	Exceção 2	Exceção 3	Grava LOG_ERRO?	Demonstrada em
FN_COBRANCA_JSON	NO_DATA_FOUND	VALUE_ERROR	OTHERS	Não — chamada por trigger; COMMIT em trigger é ORA-04092	bloco 11.2 (NO_DATA_FOUND, VALUE_ERROR)
PRC_LISTAR_COBRANCAS_JSON	NO_DATA_FOUND	VALUE_ERROR	OTHERS	Sim	bloco 11.4 (NO_DATA_FOUND)
PRC_RELATORIO_COBRANCAS	NO_DATA_FOUND	VALUE_ERROR	OTHERS	Sim	bloco 11.6 (NO_DATA_FOUND)
FN_CALCULAR_SCORE_URGENCIA	e_texto_excede_limite (grupo)	VALUE_ERROR	OTHERS	Sim	bloco 11.8 (e_texto_excede_limite)

A ASSIMETRIA, DECLARADA

Três dos quatro objetos gravam a exceção em **LOG_ERRO**; **FN_COBRANCA_JSON** não. Isso não é esquecimento: essa função é chamada **de dentro da trigger**, e gravar em **LOG_ERRO** exigiria **COMMIT** — e **COMMIT** dentro de trigger levanta **ORA-04092**, derrubando o DML de negócio que a auditoria deveria apenas observar.

O tratamento de erro dessa função é, por isso, **devolver JSON de fallback e nunca propagar** — a evidência fica no bloco 11.2, não em **LOG_ERRO**.

==== 11.13 LOG_ERRO -- excecoes registradas pelos objetos nesta demonstracao ====	
NM_PROCEDURE	NR_CODIGO_ERRO
DS_PARAMETROS	
MSG_45	

PRC_LISTAR_COBRANCAS_JSON p_id_clinica=999 NO_DATA_FOUND: clinica inexistente/inativa ou	100
PRC_RELATORIO_COBRANCAS p_id_clinica=999 NO_DATA_FOUND: clinica inexistente/inativa ou	100
FN_CALCULAR_SCORE_URGENCIA len=5000 TEXTO_EXCEDE_LIMITE: texto com 5000 chars exc	-20001
3 linhas selecionadas.	
##### FIM DA DEMONSTRACAO (BLOCO 11) #####	

Print 14 — bloco 11.13·apoio. As 3 exceções desta demonstração aparecem em **LOG_ERRO** com **NM_PROCEDURE**, **NR_CODIGO_ERRO** e **DS_PARAMETROS**. Arquivo: 14_log_erro.png

TRG_AUDITORIA_COBRANCA não entra nesta tabela — de propósito. Ver 7.2.

Nota sobre o código da 2ª Sprint

A rubrica pede "código da 2ª Sprint corrigido conforme feedback". Não há registro escrito desse feedback, então este documento **não** alega correção que ninguém pode conferir — declara o que foi de fato revisto e por quê:

1. **Consolidação do schema de 26 para 30 tabelas.** O arquivo da Sprint 1/2 (`kura_schema_final_entrega.sql`) tinha 26 tabelas; o schema real, mantido por 19 migrations Flyway, tem **30** (as migrations V15, V17 e V18 acrescentaram `INTERACAO_CANAL`, `USUARIO_CLINICA`, `SERVICO_PRECO` e `COBRANCA`). O `.sql` de entrega reflete o estado atual (V19), não o de maio.
2. **Tipos alinhados ao banco real.** O arquivo antigo tinha, por exemplo, `ST_STATUS VARCHAR2(50)` onde a migration V1 define `VARCHAR2(20)`. Cada uma das 30 tabelas foi consolidada conferindo coluna a coluna contra as migrations.
3. **Estratégia de chave primária fiel ao banco.** 26 tabelas com `DEFAULT SEQ_X.NEXTVAL`, 4 com `IDENTITY` — como está no banco, não uniformizado.
4. **Preenchimento dos RMs na capa.**

Uso de Inteligência Artificial

O grupo usou IA (assistente de código) no processo, e declara honestamente como:

- **Consolidação do schema:** o schema de 30 tabelas veio das 19 migrations Flyway do repositório `backend-tutor-java`. A consolidação foi conferida **tabela a tabela contra um mapa escrito antes** (`MAPA-CONSOLIDACAO.md`), e depois **validada por execução contra o Oracle real** — não por leitura de diff. Uma verificação automática (`SD-02-verificacao.sql`) confirmou 30 tabelas, 29 sequences, 0 objeto inválido e a contagem de colunas de cada tabela.
- **Os 5 objetos PL/SQL:** cada um foi escrito, **compilado contra o Oracle da FIAP**, e testado com um script de teste próprio (`SD-04-teste.sql ... SD-07-teste.sql`). Bugs de runtime foram encontrados e corrigidos por medição: por exemplo, um laço infinito em `FN_CALCULAR_SCORE_URGENCIA` (`INSTR` de `NULL` devolve `NULL`, e `EXIT WHEN v_pos = 0` não pega `NULL`) só apareceu ao executar, não ao ler.
- **A Função 2:** portada linha a linha do `trriage_engine.py/trriage_rules.py` da Luna, e validada contra uma **reimplementação independente** do algoritmo — 7 casos de teste, 7 idênticos.
- **A trigger:** provada por **mutação** — desabilitada, repetido o DML, confirmado que nada foi gravado; reabilitada, voltou a gravar. Não se assumiu que "compilou" significa "funciona".

Nenhum dos 5 objetos foi entregue sem execução contra o banco.

Pré-requisitos e como executar

Pré-requisitos: Oracle Database 12c ou superior (a sintaxe `DEFAULT SEQ.NEXTVAL` e `GENERATED BY DEFAULT AS IDENTITY` exige 12c+; a entrega foi feita no **Oracle 19c** da FIAP). Cliente: **SQL Developer** (recomendado — evita mojobake por `NLS_LANG` mal configurado).

Executar:

1. Abrir `2TDS PF_2026_CodigoSql_Integrantes.sql` no SQL Developer, conectado ao schema de aluno.
2. **Run Script (F5)** — não *Run Statement*.
3. O bloco 0 já faz `SET SERVEROUTPUT ON SIZE UNLIMITED` e `SET LINESIZE 400`.
4. O script é **re-executável**: o bloco 1 (limpeza) derruba, por nome, só os objetos do KURA, antes de recriar. Rodar 2× seguidas dá o mesmo resultado.

O que esperar: blocos 0–7 (estrutura + carga) rodam com **0 linhas ORA-**; blocos 8–10 compilam os 5 objetos sem erro; o bloco 11 (demonstração) exercita os 5 objetos no caminho feliz **e dispara cada exceção tratada**.

LEITURA DA SAÍDA DO BLOCO 11

As exceções são **tratadas** (não vazam), então a saída do bloco 11 pode conter a string `ORA-01403` etc. dentro do texto de `SQLERRM` que os handlers registram: isso é **evidência de que o tratamento de erro funciona**, não falha de execução.

Mapa dos prints

Print	Arquivo em docs/prints/	Origem no .sql
1	01_fn_cobranca_json_sucesso.png	bloco 11.1
2	02_fn_cobranca_json_excecoes.png	bloco 11.2
3	03_prc_listar_cobrancas_json.png	bloco 11.3
4	04_prc_listar_json_excecao.png	bloco 11.4
5	05_prc_relatorio_cobrancas.png	bloco 11.5
6	06_prc_relatorio_excecao.png	bloco 11.6
7	07_fn_calcular_score_urgencia.png	bloco 11.7
8	08_fn_score_urgencia_excecao.png	bloco 11.8
9	09_trigger_insert.png	bloco 11.10 (INSERT)
10	10_trigger_update.png	bloco 11.10 (UPDATE)
11	11_trigger_delete_old.png + 11_trigger_delete_new.png	bloco 11.10/11.11 (DELETE)
12	12_auditoria_e_contagem.png	bloco 11.12
13	13_triagem_luna_update.png	bloco 11.9
14	14_log_erro.png	bloco 11.13

Como capturar cada print: cada imagem deve mostrar **o comando e a saída juntos** (não só a saída). Capturar direto do painel *Script Output* do SQL Developer, com a janela larga o suficiente para o JSON não quebrar.